Vegas: A Domain-Specific Language for On-Chain Strategic Protocols

Elazar Gershuni
elazarg@gmail.com

May 2026

Abstract

A smart contract that implements an auction, a voting procedure, or any other multi-party economic mechanism is two things at once: a program that runs on a public, adversarially ordered execution layer, and a strategic game played by rational participants. The contract is usually written, audited, and deployed as the former; reasoning about its incentive properties is left to a parallel pen-and-paper exercise on the latter. Vegas is a domain-specific language and compiler that treats both views as outputs of a single specification.

A Vegas program describes a finite protocol in the natural sequential style of a referee: who joins, who plays what, who reveals, and who gets paid. The compiler analyses the data flow of the specification to build a finite *event graph* — a directed acyclic graph of guarded writes — and uses it as a single intermediate representation that each backend reads in turn: (i) a Solidity (and Vyper) contract whose execution scheduler is the dependency relation itself, with built-in commit-reveal protection against transaction-ordering adversaries and a timeout-driven bail-out for non-responsive participants; (ii) a Gambit extensive-form game suitable for equilibrium analysis; (iii) several auxiliary artifacts — an SMT-LIB encoding for reachability queries, a Coq/Lean witness-record encoding of the protocol's per-step constraint structure, a session-typed (Scribble) protocol description, and a Bitcoin/Lightning output for the two-party fragment. The expressive power of the language is restricted by design: only finite types, no loops, no inter-contract calls. These restrictions are what make the whole array of artifacts feasible to produce automatically from one source.

The report describes the language, the event-graph intermediate representation, the automatic commit-reveal rewriting pass, the backends, and the relationship to a companion Lean 4 library called VegasCore. We are explicit about the parts that are stable, the parts that are scoped down on purpose, and the parts that we expect to relax as the language evolves.

Contents

1 Introduction

1.1 The two-faces problem

A smart contract that hosts a sealed-bid auction is one program with two readings. Read as an Ethereum contract, it is a state machine exposing transactions: `commit(`*`hash`*`)`, `reveal(`*`bid`*`)`, `settle()`. Read as a game, it is an extensive form with players, information sets, and a payoff function. These two readings are seldom kept in sync. The contract is written, audited, deployed, and run as a state machine; the game is sketched, sometimes proved correct on paper, almost never mechanically connected to the bytecode the validators actually execute. The gap is where strategic surprises live: a sealed-bid auction whose "commit" phase happens to publish enough information to be front-run; an escrow whose "timeout" branch creates a griefing incentive that

no equilibrium analysis predicted because the model omitted the timeout; a voting scheme whose Solidity implementation reveals the tally one block earlier than the analysis assumed.

Vegas is a small programming language and compiler aimed at this specific gap. A Vegas program is a finite multi-party protocol written in the natural style of a referee: who joins, who plays what, what is hidden and revealed when, and how the pool is split at the end. Programs are short — the running examples in this report are a dozen lines each — and look more like the textbook description of a game than like a smart-contract framework. The compiler reads the program, infers a dependency relation among its actions, and emits a family of artifacts that all denote the same protocol from different viewpoints: a Solidity contract for deployment, an extensive-form game for equilibrium analysis, an SMT-LIB encoding for reachability queries, and several others. Each artifact is mechanically derived from the same source.

This report describes how that compilation is organized, what the language can and cannot express, how the central intermediate representation — the *event graph* — captures the strategic structure of a protocol, and how the per-backend lowerings are constrained so that the artifacts agree.

1.2 What Vegas is and is not

Vegas is deliberately narrow. The aim is to make the round trip from specification to deployed contract to game-theoretic analysis short and reliable for a useful class of protocols, not to be a general purpose smart-contract language. Concretely, in scope are:

- finite, terminating multi-party protocols with a single pool of locked funds;
- bounded types (booleans, finite ranges, enumerations);
- deterministic guards (`where` clauses) over prior public values and over the value being written;
- public randomness via an explicit `random` construct;
- commit-reveal patterns, with automatic insertion when the compiler detects simultaneous public moves that would otherwise be front-runnable;
- explicit operational handling of non-responsive participants (timeouts and bail-outs, with handler clauses chosen by the programmer);
- terminal allocation of the pool by a `withdraw` clause.

Out of scope, on purpose:

- unbounded data and loops — the event graph must be finite;
- ERC-20/ERC-777 token flows and multi-token games (planned, not in the current language);
- interactions with arbitrary other contracts (the protocol is a closed system; cross-contract MEV is parameter, not modelled);
- private randomness from outside oracles;
- dynamic player counts (the role set is fixed at compile time).

These restrictions are real costs. They are also what make the multi-backend story feasible: every artifact is computed by exhaustive enumeration or simple recursion over a finite graph, with no need to approximate or to choose a sound-but-incomplete encoding. A Gambit extensive-form game can be built directly from the EventGraph because the graph is finite; an SMT-LIB encoding

of "can role R reach a configuration where they get $\geq x$ wei?" is quantifier-free over a finite set of action witnesses; a Lean or Coq witness-record encoding terminates by construction. We expect the language to grow — variable deposits, tokens, bounded iteration — and we have tried to make the IR robust enough to accommodate those extensions. The current paper documents the foundations.

1.3 A first example

The following Vegas program is the complete specification of a distribution-semantics Monty Hall game. Two roles, Host and Guest, each lock 20 wei of deposit into a 40 wei pool. The Host hides the car behind one of three doors; the Guest picks a door; the Host reveals a goat door; the Guest decides whether to switch; the Host reveals the car; payouts are computed from the public transcript. If a role fails to act in time, the `or split` handler forfeits their deposit to the surviving party.

```
type door = {0, 1, 2}

game main() {
  join Host() $ 20;
  join Guest() $ 20;
  commit or split Host(car: door);
  yield or split Guest(d: door);
  yield or split Host(goat: door) where Host.goat != Guest.d;
  yield or split Guest(switch: bool);
  reveal Host(car: door) where Host.goat != Host.car;
  withdraw { Guest -> ((Guest.d != Host.car) <-> Guest.switch) ? 40 : 0;
             Host  -> ((Guest.d != Host.car) <-> Guest.switch) ? 0 : 40 }
}
```

From this twelve-line source Vegas produces, with no further input:

- a Solidity contract of roughly 200 lines that enforces the protocol step by step, uses cryptographic commitments for the Host's hidden door, and falls back to the `or split` clauses if any role times out;
- an extensive-form game (`.efg`) in Gambit's format, containing the move tree and the information sets generated by the dependency relation;
- an SMT-LIB witness-existence encoding for reachability queries;
- a Coq (Gallina) and a Lean 4 witness-record encoding of the protocol's per-step constraints — the *Diamond DAG* encoding of Section 8.

All four artifacts are produced by the current compiler and exercised by the golden-master test suite. The Solidity output is deployable to a local Anvil node; the Gambit output, fed to Gambit, returns the mixed-strategy equilibrium for Monty Hall. Nothing in the twelve-line source mentions Ethereum, cryptographic commitments, timeouts, gas, or game trees.

The example is small enough to verify by inspection that the two artifacts agree on what they say is hidden, what is public, when each player observes what, and which deviations are punished. At larger scale the agreement has to be argued, not inspected — which is what motivates the companion Lean formalization VegasCore discussed in §10.

1.4 Why an event graph

A natural alternative would be to compile the surface syntax directly to a state machine for Solidity and directly to a tree for Gambit, twice, with the two backends owning their own semantics. That is also more or less what an experienced human writes by hand when they want both views of the same protocol. Vegas does not do this for three reasons.

First, the textual order of the Vegas source does *not* encode the strategic structure. In the Monty Hall example above, the line `yield Guest(d: door)` appears below the Host's hidden `commit`. But the Guest cannot read the Host's hidden value, and the Host's commit has no data dependency on the Guest; the two actions are concurrent in the strategic sense, regardless of the order they happen to be typed in. Compiling to a tree directly would force an arbitrary ordering choice, which then has to be "undone" with information-set equivalences. Inferring concurrency from data dependencies once, in the IR, and letting every backend read it off the same graph, is simpler.

Second, the operational behaviour of a Solidity contract is more naturally described by a dependency relation than by a global step counter. The Solidity backend emits a `depends` modifier per action, the modifier gating execution on the completion flags of the action's predecessors in the event graph; concurrent actions can arrive in any order. This matches the on-chain reality (the transaction ordering is chosen by builders, not by the contract) without the contract having to encode a totalisation.

Third, the rewriting passes the compiler performs — in particular, automatic commit-reveal insertion — are dependency-graph rewrites. They are easier to state and easier to test on a graph than on the syntax tree.

So Vegas uses a single intermediate representation, the *EventGraph*, between the surface language and every backend. Each backend reads the graph; no backend reaches behind the graph to the surface syntax. Section 4 defines the EventGraph in detail; the rest of the report uses it pervasively.

1.5 Contributions and outline

The contributions of this report are pragmatic rather than foundational. We claim:

1. *A language design* (§2) that makes it feasible to describe a finite on-chain protocol in fewer than twenty lines and that compiles, without further specification, to both deployable code and a game-theoretic model.
2. *The EventGraph IR* (§4) and its associated invariants (acyclicity, commit-reveal ordering, visibility on guard reads), capturing concurrency, observability, and the protocol's information-set structure in one finite object.
3. *Automatic commit-reveal insertion* (§5): a graph-rewriting pass that detects concurrent public writes and replaces each one with a commit-then-reveal pair, with predecessors set up so that no player can observe another player's reveal before committing themselves. Within the protocol, this turns an observation-before-action vulnerability into a structural property of the IR rather than an implementation discipline.
4. *A family of backends* (§§6–8) that each consume the same EventGraph: Solidity and Vyper for EVM deployment with `depends`-modifier scheduling and bail-out subgames; Gambit for extensive-form game generation via frontier exploration; SMT-LIB for reachability queries;

Scribble for the session-typed protocol description; a Bitcoin/Lightning backend for the two-player fragment; a Coq/Lean "Diamond DAG" witness-record encoding that captures the per-step constraint bundle as a dependent type.

5. *An operational subgame layer* (§9) for liveness and griefing: every action can be guarded by a handler that fires if the actor does not respond within a deadline. The handler is a strategic choice available to the actor ("quit"), realised by the Solidity backend through a timeout-and-bail mechanism, and exposed to the game-theoretic backends as an explicit move.
6. *A bridge to a Lean 4 formalization* (§10) named VegasCore, which formalizes a core calculus over the same EventGraph carrier. Vegas and VegasCore are independent artifacts: Vegas is a compiler tool, VegasCore is a self-contained semantic construction. Where they describe the same object they use the same name; where they diverge, the divergence is recorded.
7. *An example library* (§11): thirty-nine Vegas specifications covering classical mechanisms (Vickrey auction, Prisoners' Dilemma, Centipede), Ethereum patterns (escrow, micropayment channel, governance voting), and adversarial constructions (front-running setups, hidden-reserve auctions). Each example is compiled to every applicable backend.

The report proceeds in roughly the order of the compiler. §2 introduces the surface language and walks through one example end-to-end. §3 fixes the overall pipeline. §4 defines the EventGraph and its invariants. §5 describes the automatic commit-reveal pass. §§6–8 cover the backends. §9 describes the liveness/quit-handler layer. §10 maps the construction onto the companion Lean formalization. §11 reports on the example suite. §§12–13 close with related work and a frank discussion of current limitations and natural extensions.

2 The Vegas Language

This section introduces the surface language by example, fixes intuitive semantics for each construct, and gives the small grammar in BNF. The next section traces what happens to a program once the compiler takes it apart.

2.1 Programs and roles

A Vegas program declares optional type aliases and macros, then a single top-level `game` block. The block names the roles, opens a `join` phase that locks each role's deposit into a common pool, and proceeds with a sequence of action statements until a `withdraw` clause splits the pool.

```
type bid = {0 .. 10}

game main() {
  join Seller() $ 0
       Bidder() $ 100;
  // ... statements ...
  withdraw { Seller -> ...; Bidder -> ... }
}
```

Role identifiers start with an uppercase letter; field names with lowercase. A field is always identified relative to its owning role: `Bidder.b` denotes the field `b` of role `Bidder`. Each field has at most one writing action; the language has no explicit reassignment.

2.2 Action statements

There are four action kinds. Each writes one or more fields owned by its actor.

`join`. Initial deposit and admission. The `join` phase appears once, at the top of the game. A role listed in the join phase is committed to the game and has its deposit locked into the pool.

`yield`. A public move. The actor declares a value of a given type:

```
yield Guest(d: door);
```

After this action fires, `Guest.d` is visible to every role. A `yield` can carry a `where` guard, e.g. `yield Host(goat: door) where Host.goat != Guest.d`; this guard is enforced when the move is played.

`commit`. A private move. The actor declares a value that is held cryptographically (in the deployed contract: under a hash with salt) until a later `reveal`:

```
commit Host(car: door);
// ...
reveal Host(car: door) where Host.goat != Host.car;
```

A `where` clause on a `commit` is *deferred*: it is checked at the matching `reveal`, not at the commit, because the committed value is not yet observable.

`reveal`. Make a previously committed field public. Every `commit` is paired with at most one `reveal` for the same field; before reveal, only the committing actor knows the value (operationally: the contract has only the hash).

Simultaneous moves. Multiple roles can appear in a single statement; they are then deemed simultaneous:

```
yield or split A(c: bool) B(c: bool);
```

This is exactly the situation that would invite front-running if written naively, and is the canonical input to the automatic commit-reveal pass described in §5.

2.3 Guards and views

A guard is a `where` clause attached to an action. Its scope is the snapshot of fields visible *just before* the action fires, plus the field being written. Two distinctions matter.

Self-only versus contextual guards. A guard that references only the field being written (`where x != 2`) constrains the domain of that field; a guard that references prior fields (`where Host.goat != Guest.d`) constrains the legality of a move based on the public history. Both are allowed. Both are statically required to reference only fields the actor can observe at this point.

Public versus private fields. An actor cannot observe another actor's hidden field; therefore a guard cannot read it. The compiler rejects programs that try (it cannot enforce a constraint nobody can check). When the guard is on a `commit`, however, the actor's own freshly written field is private to them: the constraint is recorded and re-checked when the value is revealed, so that any later observer can verify the constraint at reveal time.

2.4 Quit handlers

Real protocols have to cope with a player who stops responding. Vegas treats this as a strategic move named `Quit`: at any point where a role is expected to act, the role can choose to abandon the game. What happens next is specified by an *or-handler* on the action:

```
yield or split Guest(d: door);
yield or burn A(c: bool);
yield or null Odd(c: bool) Even(c: bool);
```

The handlers do the following:

- `split` — the quitter's deposit is split among the surviving roles. In a two-player game, this is forfeit.
- `burn` — the quitter's deposit is destroyed (or sent to a designated burn address).
- `null` — the missing value becomes `null` (typed `opt T`); downstream actions and the `withdraw` clause are required to handle the optional case.

Operationally the handler fires when a timeout elapses without the expected action. At the game-theoretic level, `Quit` is just another move available to the actor; equilibrium analyses see it explicitly. §9 treats the subgame structure in detail.

2.5 Withdraw and conservation

A game terminates by reaching a `withdraw` clause, which allocates the entire locked pool to roles. The total must match the pool exactly: there is no implicit dust and no implicit burn. `withdraw` can be conditional:

```
withdraw (A.c && B.c) ? { A -> 100; B -> 100 }
      : (A.c)       ? { A -> 200; B -> 0 }
      : (B.c)       ? { A -> 0;  B -> 200 }
      :               { A -> 90; B -> 110 }
```

The compiler statically checks conservation against the constant total deposit; payoff expressions may reference any public field.

2.6 Types

Vegas types are deliberately small. The base types are:

- `bool`;
- finite enumerations (`type door = {0, 1, 2}`) and finite ranges (`type bid = {0..10}`);
- `address` (used only in joins and metadata; not first-class in payoff expressions);
- `int` (mapped to a 256-bit integer in the EVM backends; the Gambit backend rejects unbounded `int` in moves because it needs a finite move alphabet).

A field declared with the `hidden` prefix (`yield Host(car:  hidden door)`) is private to its owner until revealed. An `opt T` type marks a value that may be missing (typically because the actor took the `Quit` branch); the language requires `withdraw` expressions to handle the `null` case explicitly.

2.7 Macros

Macros are hygienic expression-level abbreviations, inlined before the IR is built. They simplify reasoning about non-trivial mechanisms; Vickrey auctions, for instance, are easier to read with a `secondMax` macro.

```
macro max2(a: bid, b: bid): bid = (a >= b ? a : b);
macro secondMax(a: bid, b: bid, c: bid): bid =
    (a == max3(a,b,c)) ? max2(b, c)
  : (b == max3(a,b,c)) ? max2(a, c)
  :                      max2(a, b);
```

Macros expand syntactically; recursion is disallowed. Macro expansion is purely textual at the expression level: macros do not introduce new actions or rewrite the action sequence.

2.8 A walked example: Vickrey auction

Putting the constructs together, here is a three-bidder sealed-bid second-price (Vickrey) auction with a fixed-domain bid type. The `yield or split` statement carries three actor heads, declaring the three bids as simultaneous.

```
type bid = {0 .. 2}

macro max2(a: bid, b: bid): bid = (a >= b ? a : b);
macro max3(a: bid, b: bid, c: bid): bid = max2(max2(a, b), c);
macro secondMax(a: bid, b: bid, c: bid): bid =
    (a == max3(a,b,c)) ? max2(b, c)
  : (b == max3(a,b,c)) ? max2(a, c)
  :                      max2(a, b);
macro isWinner(myBid: bid, b1: bid, b2: bid, b3: bid): bool =
    (myBid == max3(b1, b2, b3));

game main() {
  join Seller() $ 100 B1() $ 100 B2() $ 100 B3() $ 100;

  yield or split
    B1(b: bid)
    B2(b: bid)
    B3(b: bid);

  withdraw {
    Seller -> 100 + secondMax(B1.b, B2.b, B3.b);
    B1 -> isWinner(B1.b, B1.b, B2.b, B3.b)
           ? (100 - secondMax(B1.b, B2.b, B3.b)) : 100;
    B2 -> isWinner(B2.b, B1.b, B2.b, B3.b)
           ? (100 - secondMax(B1.b, B2.b, B3.b)) : 100;
    B3 -> isWinner(B3.b, B1.b, B2.b, B3.b)
           ? (100 - secondMax(B1.b, B2.b, B3.b)) : 100;
  }
}
```

The compiler does not assume the user has solved front-running: it reads three simultaneous public moves and rewrites the EventGraph so that each bidder commits to a hash first and reveals later, with the reveal of any bidder gated on the commits of the other two. The resulting Solidity

contract implements a sealed-bid auction; the resulting Gambit game has each bidder choose a hidden value in a common information set. Both come from the same source.

2.9 Grammar

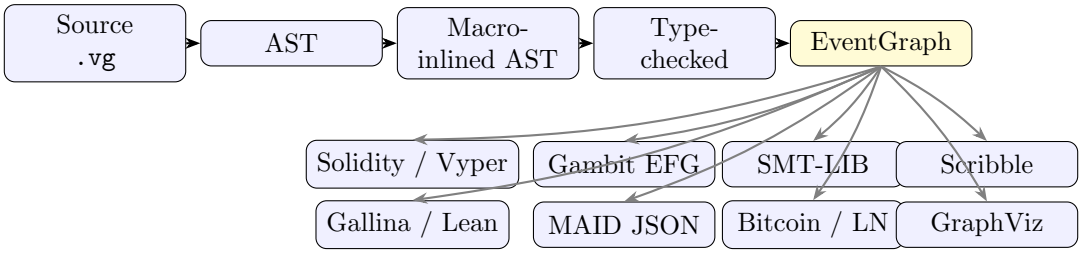
For reference, the full surface grammar lives in the ANTLR file `Vegas.g4` at the project root; the salient productions are roughly:

program ::= (*typeDec* | *macroDec*)* `game` *ident* `()` { *ext* }
ext ::= **join** *query*+ `;` *ext* | (**yield** | **reveal** | **commit** | **random**) `or` *handler query*+ `;` *ext*
| **withdraw** *outcome*
query ::= *role* `(` *paramDec** `)` ($ *int*)? (**where** *exp*)? (`||` *queryHandler*)?
handler ::= `split` | `burn` | `null`.

The grammar is unsurprising; what makes Vegas work is the analysis done in the next several sections.

3 Compilation Pipeline

The compiler is a straight pipeline from source to artifacts:



The frontend (`vegas/frontend/`) parses with an ANTLR-generated parser, inlines macros, and runs a four-pass type checker that validates type aliases, macro signatures, the protocol's `join/yield/reveal/commit/withdraw` structure, and the visibility discipline of guards. The frontend then lowers to an intermediate representation `GameIR`, which packages a finite EventGraph together with a per-role payoff expression and the chance-role set. The EventGraph is the spine: every backend reads it directly and emits its own artifact without revisiting the surface syntax.

Two transformation passes operate on the IR. The first builds the EventGraph from typechecked AST: it groups simultaneous moves into phases, infers dependencies (data, commit-reveal, phase-order), and computes per-node visibility (COMMIT, REVEAL, or PUBLIC). The second is the *commit-reveal expansion* pass (§5), which detects concurrent public-write nodes (the "risk partners") and rewrites each such node into a commit-then-reveal pair, with predecessors arranged so that no reveal is enabled while a sibling's commit is still unfinished.

After the rewrite, the EventGraph satisfies three invariants: acyclicity; commit-before-reveal (for each pair of COMMIT/REVEALnodes on the same field, the commit dominates the reveal in the DAG); and read-visibility (every field referenced by a guard is either public at the action's point in the DAG, or has already been revealed in a predecessor). The graph constructor refuses to return an EventGraph that violates these invariants; backends can assume them.

Backends fall into two groups. The *operational* backends — Solidity, Vyper, Bitcoin/Lightning — emit an executable artifact that implements the protocol on its target runtime. The *analytical* backends — Gambit, SMT-LIB, MAID, Scribble, Gallina/Lean — emit a non-executable

representation suitable for external tools (Gambit for equilibrium computation, Z3 for reachability, Coq/Lean for proof, etc.). Both groups consume the same EventGraph. The next several sections walk through each in turn.

4 The EventGraph

This section introduces the EventGraph: the finite, visibility-aware directed acyclic graph that all Vegas backends read. The graph is the structural core of the language — everything from front-running mitigation to information-set construction to bail-out subgames is expressed as a property of, or a rewrite on, this graph.

4.1 Nodes

A node of the EventGraph represents one statement in the protocol: a `join`, a `yield`, a `commit`, a `reveal`, or a `random`. Each node is identified by a pair $\mathit{NodeId} = (\mathsf{Role}, n)$ where Role is the actor and n is an integer giving the node's source position. The pair is stable: it is fixed at IR construction time and used throughout backends as a deterministic identifier.

To each node the IR attaches a payload of three pieces:

- the *specification* (`NodeSpec`): the parameter list, any deposit (for `join` nodes), and the guard expression;
- the *structural metadata* (`NodeStruct`): the owner, the set of written fields, the visibility of each written field (one of COMMIT, REVEAL, PUBLIC), and the set of fields the guard reads from prior nodes;
- a derived *kind* (`Visibility`), which is the "dominant" visibility of the node's writes — COMMITfor hidden writes, REVEALfor matching reveals, PUBLICfor plain writes.

The split between `NodeSpec` and `NodeStruct` is mainly an engineering choice: backends that need only structural information (the Gambit and Scribble backends, primarily) can ignore the spec.

4.2 Edges and dependency inference

Edges in the EventGraph represent control or data dependencies. The IR builds them by analysing the typechecked AST. An edge $A \to B$ means B cannot fire until A has fired. The compiler installs an edge $A \to B$ when any of the following holds:

- *Phase order.* If A and B are in adjacent source phases (consecutive top-level statements), $A \to B$. This preserves the textual ordering at the granularity of phases, which is what the programmer can rely on.
- *Guard read.* If B's guard reads a field f, and the latest writer of f before B is A, then $A \to B$. The guard cannot evaluate until the value is available.
- *Commit before reveal.* If B is a REVEALof field f and A is the COMMITof f, then $A \to B$. The reveal must come after its commit.

Within a single phase, multiple roles can act simultaneously (they appear in the same `yield/commit` statement). Their nodes share predecessors and are not connected to each other: they are concurrent in the dependency graph, which is what justifies treating them as simultaneous moves in the strategic analysis.

4.3 Visibility

Each written field has a visibility tag that records what an observer learns from the corresponding write. The three tags are:

public. The value is public from the moment of the write. Any later node may read it.

commit. The value is hidden. Operationally, only its cryptographic commitment is public; the actor knows the value; no other actor learns it from this node.

reveal. A previously committed value is now public. The reveal is paired with the unique prior commit of the same field.

The IR ensures that visibility is consistent with the dependency relation. In particular:

- every REVEALis reachable from its matching COMMIT;
- every field referenced by a guard at node B has either been written PUBLICby some predecessor of B, or has been revealed by some predecessor of B (so its value is known publicly at B);
- the owner of a node always sees their own writes (an actor may guard their action on a field they have just decided — this is how self-only guards like `where x != 2` are modelled).

If any of these conditions fails, the EventGraph constructor refuses to build the graph; the compiler surfaces this as a static error. Backends therefore work in a setting where the information flow is fully consistent.

4.4 Configurations and frontier

A *configuration* of the EventGraph is a partial assignment from nodes to outcomes (one outcome per node: a tuple of values for the node's written fields, or `Quit`). A node is *enabled* in a configuration when all its predecessors have been assigned and its guard, evaluated on the configuration's public writes plus the actor's local view, is true. The set of enabled, unassigned nodes is the configuration's *frontier*.

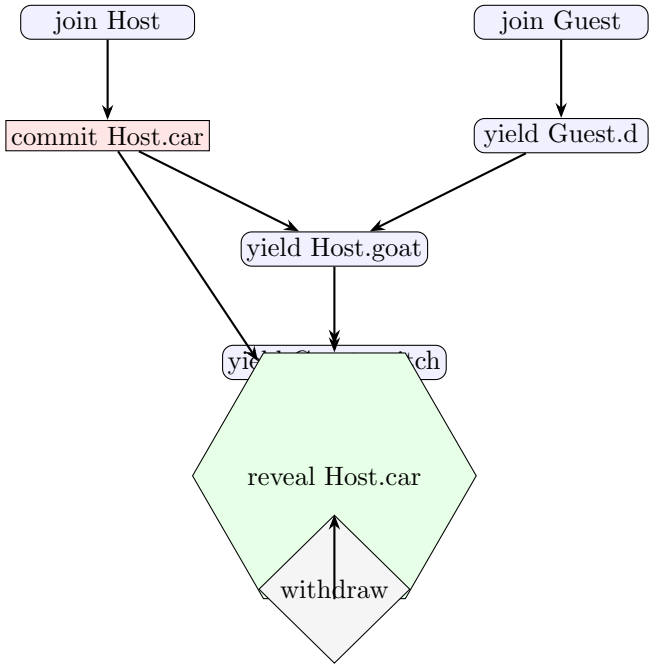
Two properties of the frontier matter for the backends:

- *Concurrency.* If two distinct nodes A, B are on the same frontier — neither reaches the other in the DAG — scheduling A before B versus B before A produces the same configuration after both have fired. This is what allows Solidity to accept the actions in any order and Gambit to model them as simultaneous.
- *Information barrier.* If a node B is a REVEAL, every node on which it depends has already fired by the time B is enabled. In particular, in any configuration where B's match COMMITis unfinished, B is not on the frontier. This is the structural counterpart to the commit-reveal insertion pass: once a player can reveal, every other player has already committed; no player can condition a commit on another's revealed value.

The compiler computes the frontier on demand with a small `FrontierMachine` that maintains the set of unresolved nodes along with the current frontier; resolving the frontier advances both sets in one step. The Gambit backend explores the resulting state space depth-first (§7), treating each frontier resolution as one strategic round.

4.5 A worked graph

The Monty Hall example from §1.3 produces the EventGraph below. The two `join` statements share one source phase (the source has `join Host()...Guest()` combined in a single top-level statement) and are therefore concurrent; the remaining statements are separate phases and pick up phase-order edges between them, so the rest of the graph is a chain. Squares are COMMITwrites, hexagons are REVEALwrites, ovals are PUBLICwrites, and the diamond is the terminal `withdraw` sink. The commit-reveal expansion pass (§5) does nothing here, because none of the public yields are concurrent with each other.



A few facts to read off this picture:

- `join Host` and `join Guest` are the only concurrent pair: they share a phase, neither reaches the other, and the Solidity contract accepts the corresponding two transactions in either order.
- `reveal Host.car` is enabled only after Guest has switched. In any configuration where the reveal is on the frontier, every prior decision has been made. Guest cannot switch *after* seeing the car location, and Host cannot revise the car location.
- `yield Host.goat` reads `Guest.d` (the guard `Host.goat != Guest.d`), which would force the order `yield Guest.d` → `yield Host.goat` even if the two statements were in the same phase. Here, phase order contributes the same edge.

Sequential graphs like this are the typical shape of textbook turn-taking games; concurrency-driven rewriting becomes interesting when multiple roles *share* a yield statement, as in the Vickrey example treated in the next section.

4.6 Visibility-aware perfect recall

For game-theoretic analysis, two players are in the same information set at a node B iff they have observed the same sequence of public events leading to B. In the EventGraph this becomes a structural notion: the set of fields observable by role R at node B is

$\text{obs}_R(B) = \{ f \mid f \text{ is written by some predecessor } A \text{ of } B, \text{ and either owner}(A) = R \text{ or vis}(A, f) \in \{\text{PUBLIC}, \text{REV}$

This is the projection used by the Gambit backend to compute information sets and, in a closely related form, by the MAID and Scribble backends. An IR helper `observableFieldsAt(B)` computes it; the formula is the only place where "observability" is defined.

4.7 Implementation notes

The EventGraph class wraps a generic `Dag<NodeId>` with the per-node payload map and a precomputed reachability relation. Reachability is computed once when the graph is built and reused for all queries; building it eagerly is the right tradeoff in this setting because the graph is small (forty-some nodes for the largest example) and the queries are many (`reaches`, `ancestorsOf`, `canExecuteConcurrently`).

The constructor takes a node set, a per-node predecessor map, and a per-node payload map, and returns either an EventGraph or `null`; the latter happens when an invariant is violated. Three internal validators run inside the constructor:

- acyclicity, via topological sort;
- commit-reveal ordering, by checking that every REVEALhas a matching COMMITamong its ancestors;
- read-visibility, by checking that every field appearing in a guard at node B is either written PUBLICbefore B, or has been revealed before B, or is being written by B itself.

The corresponding error in the frontend is a static error, with a source span pointing back to the offending `where` clause or `commit/reveal` pair. This is the same set of checks a human reviewer would perform when reading a hand-written commit-reveal contract — the IR just performs them systematically.

A note on the read-visibility check: in addition to fields written PUBLICor revealed by a predecessor, the validator also accepts a guard read of a field that the actor itself committed earlier on the same DAG path. An owner "sees their own writes" is therefore a slightly stronger statement than the bullet above admits: it includes their own *committed* fields as well as the public/revealed ones. In practice the typechecker's pre-IR desugaring strips fields written by the current node from the guard's scope, so this branch matters mostly for guards that reference an actor's earlier private writes.

5 Automatic Commit-Reveal Insertion

This section describes the EventGraph rewrite that protects simultaneous public moves from transaction-ordering adversaries. The pass is small but the property it secures is one of the most visible payoffs of using Vegas: a programmer who writes "two players `yield` simultaneously" gets, deployed to chain, the commit-reveal protocol that the textbook would have insisted on.

5.1 The problem

Consider the two-line specification

```
yield A(x: bool);
yield B(y: bool);
```

At the EventGraph level these are two PUBLIC writes on concurrent nodes. Read as a game, the analysis is unambiguous: A and B move simultaneously without observing each other. Read as a Solidity contract, the implementation has to be cautious. If A's transaction is in the public mempool, B can read `x` before deciding `y`. An ordering adversary (a builder, or a competing searcher) can promote the read-then-write transaction. The specification's notion of simultaneity is broken by the deployment target's notion of transaction ordering.

This is a specific subclass of MEV — *observation-before-action* on simultaneous moves — and the canonical mitigation is well known [3, 4, 22]: each role first commits to a hash of their move, and only after all roles have committed does anybody reveal. Vegas applies the mitigation automatically, by rewriting the EventGraph.

5.2 Risk partners

Two nodes are *risk partners* when both are pure-public writes, neither reaches the other in the dependency DAG, and they are distinct. This is precisely the configuration in which the ordering adversary problem arises. The pass identifies the set of nodes that have at least one risk partner — call this set Split — and rewrites every node in Split.

Pure-public writes (every visibility tag is PUBLIC) are the only candidates for rewriting. Nodes that already commit (visibility COMMIT) need no rewriting; their reveal partner is already present. Nodes that themselves have a guard reading another actor's private value cannot arise (the type-checker rejects such reads). And nodes that are in a strict dependency relation with their potential partner are not concurrent, so the ordering adversary has no informational asymmetry to exploit.

5.3 The rewrite

For each $a \in$ Split the pass introduces two fresh nodes:

- a *commit node* a^c, replacing a as the new owner of the deposit (if a was a `join` step) and of the original predecessors of a, but with all of a's PUBLICwrites retagged as COMMITand with a trivial guard (anything is allowed to commit);
- a *reveal node* a^r, carrying a's original guard, with all the same writes retagged as REVEAL.

Edges are wired so that

- a^c has the same predecessors a originally had;
- a^r has predecessors $\{a^c\} \cup \{b^c : b \in \text{partners}(a)\} \cup \text{pred}(a)$;
- every node that previously depended on a now depends on a^r instead.

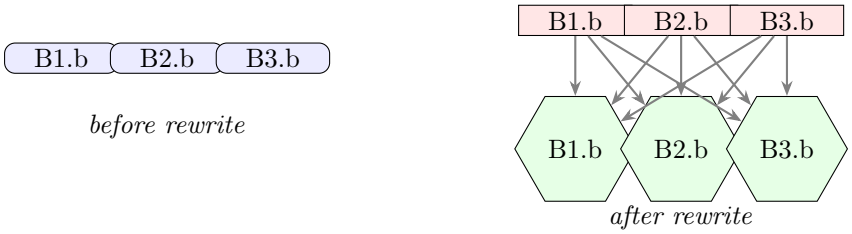
The second clause is the informational barrier: a^r cannot fire until *all* of a's risk partners have also committed. After the rewrite, no REVEALnode is on the frontier of any configuration in which a sibling COMMITis still unresolved. Section 4.4 called this the structural information barrier; the rewrite is what guarantees it for any group of originally-simultaneous public moves.

5.4 Illustration

The bidder fragment of the Vickrey auction (§2.8) is the cleanest case. The source

```
yield or split B1(b: bid) B2(b: bid) B3(b: bid);
```

produces, before the rewrite, three concurrent PUBLICnodes (left) and after the rewrite, three commit nodes followed by three reveal nodes that all wait for the full commit phase (right).



After the rewrite, each bidder's reveal depends on every bidder's commit. Reading off the new graph: no reveal is enabled on a frontier where any commit is missing, so no bidder learns another bidder's value before they themselves have committed.

5.5 What changes for the backends

Every backend sees the rewritten graph, not the original. The operational backends (Solidity/Vyper) therefore emit a commit-then-reveal pair of contract functions per rewritten action; the Solidity backend uses the hidden write's payload as the input to a hash and stores the hash, then on reveal verifies the hash and assigns the public field. The Gambit backend reads the rewritten graph as a sequence of two simultaneous-move rounds (commits, then reveals); information sets are computed from the new graph in the usual way, and they correctly reflect the informational barrier. No backend-specific commit-reveal logic exists: the property is paid for once, in the IR, and consumed everywhere.

5.6 What the rewrite does *not* do

A few honest caveats.

Cryptographic strength is not the IR's concern. The rewrite stipulates *that* a hidden value is committed before the public reveal; it does not specify the cryptographic primitive. The Solidity backend chooses a domain-separated keccak with a random salt and a per-game tag; a different backend (Bitcoin) uses its native opcodes. The choice is the backend's; the IR-level property holds for any binding, hiding commitment.

The rewrite does not address adaptive adversaries elsewhere. An adversary can still censor a player's transaction to force a timeout; the bail-out subgame (§9) makes this a strategic choice with explicit payoff, so the game model sees it, but censorship-resistance per se is not what the commit-reveal pass provides.

The rewrite does not enlarge the set of safe patterns. Only *originally-concurrent* public writes are rewritten. A public write that is in strict order with all other public writes is not at risk and stays a single node. This is intentional: there is no point in adding round-trips for moves that the dependency relation already serialises.

5.7 Implementation

The rewrite lives at `ir/EventGraph.kt` as a static method `expandCommitReveal` on the `EventGraph` class. It runs once, immediately after the initial IR construction, before any backend gets to read the graph. The implementation is about sixty lines of Kotlin and is reachability-driven: it iterates over the existing nodes, builds the risk-partner relation by checking $\neg(\text{reach}(a, b) \vee \text{reach}(b, a))$ for each pair of pure-public nodes, and then performs the local rewrite for each risk-partner-bearing node, allocating fresh node ids above the current maximum. The resulting graph is re-validated by the EventGraph constructor (it must still be acyclic, the new COMMIT-REVEALpairs must be in order, and the read-visibility discipline must be preserved); the rewrite is constructed so that all three properties are preserved by inspection, and the post-rewrite validation is mostly defensive.

6 The Solidity and Vyper Backends

The EVM backends translate the EventGraph into a deployable smart contract. Both languages target the same compilation pass via a shared EVM intermediate representation (`evm.EVMIR`); only the final pretty-printer differs. This section walks through the Solidity output and notes the few divergences for Vyper. We are careful to describe the actual generated code, not an idealised version of it: the bail-out mechanism in particular is simpler than the language-level story might suggest, and a few of its consequences belong in the open-questions list rather than the guarantees list.

6.1 Threat model

The compiled contract is meant to survive a transaction-ordering adversary — a builder or competing searcher who can choose which pending transactions to include and in what order — and to remain liveness-preserving when a participant stops responding. It is not designed against:

- *Censorship.* An adversary who indefinitely excludes a participant's transactions is modeled as forcing a timeout, which marks the participant as `bailed` and triggers the language-level quit handler.
- *Cross-contract interactions.* The contract is a closed system; no calls into other contracts are emitted (the only outbound action is a direct ETH transfer to a registered participant address during withdrawal).
- *Gas-cost games.* Gas is a real-world cost that affects which moves are rational, but the analytical backends ignore it. Bounded per-action gas envelopes (e.g. via GASTAP [1]) would be needed to bring gas into the strategic model; this is planned but not implemented.
- *Cryptographic primitives.* The contract relies on keccak-256 being binding and hiding. Domain separation and per-deployment binding are in the hash composition (§6.4), but the strength of the primitive itself is taken as given.

The remainder of this section describes the construction; the "honestly imperfect" parts of the design (especially the single global timeout window) are flagged inline.

6.2 Storage layout

The contract's storage is derived directly from the EventGraph. There are five families of slots.

Per-node completion. A two-level mapping `actionDone[role][actionId]` records, for each role and each node, whether the node has fired. A symmetric mapping `actionTimestamp[role][actionId]` stores the block timestamp at which it fired. Node ids are integers assigned by a deterministic topological linearisation of the EventGraph; one `uint256 constant ACTION_`*`Role`*`_`*`n`* is emitted per node. The integer *n* is the node's source position (making the constant name human-readable), while the assigned constant *value* is its topological index.

Role registry. A mapping `roles[address] => Role` associates EOAs with the roles they have joined, and a per-role `address_`*`Role`* slot stores the registered address. A small `enum Role` carries one constructor per role plus a `None` sentinel.

Game state. For each *written field* in the EventGraph the contract stores the field's value. A field with PUBLICvisibility uses a slot of the field's native type (`int256`, `bool`, etc.); a field with COMMITvisibility uses a separate `bytes32` slot for the commitment hash, with the revealed value filled in by the matching reveal action. Each value slot is shadowed by a `done_`*`slot`*`: bool` flag.

Per-role bookkeeping. Three booleans per role: `done_`*`Role`* (the role has joined), `claimed_`*`Role`* (the role has claimed its terminal payout), and a private `bailed[`*`Role`*`]` (the role has been marked inactive after a timeout).

Game-wide constants. A constant `TIMEOUT` sets the inactivity window, in seconds (the default is 24 hours). A constant `COMMIT_TAG` (a hashed domain-separation tag) and the contract's deployed address are folded into every commitment hash.

6.3 Modifiers and the depends pattern

The contract has no global step counter. Each per-node function is guarded by a `depends` modifier per predecessor. The modifier itself does two things: it advances the contract's inactivity-timeout state (potentially marking some role `bailed`), and it conditionally requires the predecessor to have completed. Read top to bottom:

```
modifier depends(Role role, uint256 actionId) {
    _check_timestamp(role);
    if (!bailed[role]) {
        require(actionDone[role][actionId], "dependency␣not␣satisfied");
    }
    _;
}
```

The body invokes `_check_timestamp(`*`role`*`)`, which marks *role* as `bailed` iff the current `block.timestamp` exceeds a shared global high-water mark `lastTs` by more than `TIMEOUT`; otherwise it leaves the state unchanged. If *role* is not bailed, the predecessor's completion flag is required; if it is bailed, the `require` is skipped and the dependent action proceeds. Two additional modifiers complete the gating: `action(role, id)` asserts that the action has not yet fired, marks it done, runs the body, and updates `actionTimestamp` and `lastTs`; `by(role)` asserts that the calling EOA holds the given role and is not itself bailed.

A typical per-node function looks like:

```
function move_Guest_3(int256 _d)
    external
```

```
    by(Role.Guest)
    depends(Role.Host, ACTION_Host_0)
    depends(Role.Guest, ACTION_Guest_1)
    depends(Role.Host, ACTION_Host_2)
    action(Role.Guest, ACTION_Guest_3)
{
    require(_d >= 0 && _d <= 2, "domain");
    Guest_d = _d;
    done_Guest_d = true;
}
```

Every emitted action is named `move_Role_n` where n is the source position. The function's dependency list is exactly the predecessor set of the node in the EventGraph (after the commit-reveal expansion). Concurrent nodes share the prefix of the chain up to the divergence point and are otherwise free to be called in any order.

6.4 Commits and reveals

Every COMMIT-tagged write becomes a function that takes a `bytes32` commitment and stores it; every REVEAL-tagged write takes the cleartext value plus a salt, recomputes the commitment, and asserts equality with the stored hash before performing the public write. The commitment hash is composed as

$$\mathit{keccak256}\big(\text{abi.encode}(\text{COMMIT\_TAG},\ \text{address(this)},\ \text{role},\ \text{actor},\ \mathit{keccak256}(\text{payload}))\big).$$

The pieces serve distinct roles. The `COMMIT_TAG` provides cross-protocol domain separation. `address(this)` ties the commitment to this particular deployment, preventing replay across copies of the same protocol. The `role` identifier defends against role-swapping replays within a single deployment; the `actor` address binds the commitment to the EOA that posted it. The payload is pre-hashed before being folded in, so the outer hash works over a fixed-size input regardless of the parameter list.

Vyper emits the analogous construction using the language's `assert` idiom and slightly different storage syntax. The generated bytecode is comparable.

6.5 The single-window timeout and its consequences

The timeout mechanism is simpler than the language-level story might suggest, and it is worth describing exactly.

The contract maintains a single global `lastTs` slot, updated to `block.timestamp` on every successful action. Whenever a modifier is entered, `_check_timestamp(role)` is called: if `block.timestamp > lastTs + TIMEOUT`, the named role is marked `bailed` *and* `lastTs` is reset to the current timestamp. In effect, the contract has one sliding inactivity window shared across all participants: if no action has happened anywhere for `TIMEOUT` seconds, the next caller can mark *some* role as bailed and the window restarts.

This has two non-obvious consequences worth flagging:

- *The role that gets marked is the role named in the caller's modifier.* `depends(Role.Host, ...)` on a window expiry marks `Host` as bailed; `by(Role.Guest)` on a window expiry marks `Guest`. This is approximately right — the function call points at a role expected to have

acted — but it is not the same thing as "the unique unresponsive role"; the attribution is structural rather than analytical.

- *Bail is one-way.* Once `bailed[R] = true`, the `by(R)` modifier rejects any subsequent action from `R`. A role that goes offline briefly and reconnects after the window expires has lost the right to play. The language-level treatment of `Quit` as a definitive move is the source of this design; it is operationally simple, but it does mean that a fair-playing role with a flaky connection is at risk in long protocols.

When a dependent action runs against a bailed predecessor, the `depends` modifier skips its `require`, but it does *not* fabricate writes for the missing fields. Their value slots and their `done_` flags remain in their default state (zero, and `false` respectively). Downstream code that needs to distinguish "never written" from "written to the default value" relies on the language's `opt T` typing: a field declared with an `or null` handler is typed `opt T`, and the `withdraw` clause is required to inspect its `done_` flag.

The fair-play guarantee one would like — "every role can drive the contract from any reachable configuration to a terminal allocation" — is plausible for protocols whose `withdraw` clause is total on the public state (including `done_` flags), but it is not proved. Stating and proving it under the explicit execution model is the central refinement-theorem target of the ongoing VegasCore work; today, the test suite checks that the `withdraw` clause type-checks against the optional fields it must handle, and that hand-played example traces reach a terminal allocation, but no machine-checked proof is shipped.

6.6 Withdraw and payout

The terminal `withdraw` clause is compiled into one entry point per role. Each role calls its own `withdraw_Role()` to receive their share, guarded by a `claimed_Role` flag to prevent double-payment. The payout expression is lowered to EVM IR by direct structural translation; the body computes the expression against the public state (including `done_` flag inspection for optional fields) and transfers the resulting amount to the role's stored address using a low-level `call` with the computed value.

Conservation is checked statically against the deposit total during compilation (symbolic checks where possible; exhaustive enumeration for the finite-type case). The contract therefore does not need to verify at runtime that all role payouts sum to the pool.

6.7 Things the backend deliberately leaves alone

Several real concerns are not addressed by the Solidity emission because they are scoped out of the language. Gas costs as a strategic factor; mempool-level censorship beyond what the bail mechanism handles; cross-contract interactions and oracle reads; admin keys, upgrade proxies, and pausable patterns. Each is a deliberate restriction in the same vein as the language's bounded-types-only stance: drawing the boundary tightly is what makes the analytical backends' outputs faithful to the deployed code, and what makes "the same source for two views" a defensible claim.

7 The Gambit Backend

The Gambit backend turns the EventGraph into an extensive-form game in Gambit's `.efg` format [19]. Once written, the file can be loaded directly into Gambit (or OpenSpiel [16], via standard

adapters) to compute Nash equilibria, dominant strategies, or any other property Gambit knows how to check. This section describes how the backend constructs the tree from the graph.

7.1 Frontier exploration

The backend is a depth-first frontier expander. At a configuration C, it enumerates the frontier, groups its nodes by owning role, and for each role that has at least one node on the frontier it generates a Gambit decision node. Multiple roles may have nodes on the same frontier; they are the simultaneous-move case (§4.4). Gambit's `.efg` format does not have a primitive "simultaneous block": simultaneity is expressed by nesting the per-role decision nodes in a canonical role order and arranging the information sets so that each role's information set hides the other roles' choices at the same frontier. The strategic content is in the information sets, not the textual nesting order.

For each role, the actions are the products of values its nodes may take. Bounded-type fields have an explicit finite alphabet (e.g. `door = {0,1,2}` has three values); the `Quit` move is added as an extra alphabet symbol when the role's node has a quit handler. A node performing a hidden commit has its commit parameter listed as "Hidden(*value*)" rather than the cleartext value: from the perspective of the game, the value matters but the observer does not see it. When the corresponding reveal node fires, the resolved value collapses "Hidden(...)" to a concrete number.

For each combination of role moves at the current frontier, the backend computes the resulting configuration, recurses, and emits the subtree. When the frontier is empty — the configuration has finished every node — the `withdraw` clause is evaluated to produce a terminal node carrying the payoff vector.

7.2 Information sets

A player's information set at a decision node is the set of histories they cannot distinguish, given what they have observed. The backend computes this directly from $\mathrm{obs}_R(B)$ (§4.6): two nodes are in the same information set for role R iff the projections of their pre-state to R-observable fields are equal.

In practice, hidden commits create the information sets that matter. In the Vickrey example, each bidder's reveal node is at a depth where the other bidders' commitments have fired but their values have not been revealed; the reveal node therefore sits in an information set whose members differ only in the other bidders' hidden values. This is the correct strategic representation of a sealed-bid auction.

7.3 Pruning

Two simple simplifications are run on the produced tree before emission.

Unreachable subtrees are dropped. When a guard rules out a move (the guard evaluates to false for the would-be combination of values), no edge is emitted; this is straightforward as long as the guard's free variables are all evaluated at the decision node's information set.

Trivial continuations are collapsed. When a player has exactly one legal move at a decision node (often because `Quit` is the only choice on a degenerate branch), the backend emits a single edge and proceeds. This keeps the tree size manageable on examples with many forced timeouts.

7.4 Limits

A few characteristics of the backend should be acknowledged.

Tree size blows up with the type universe. The backend enumerates the cross-product of move alphabets at each frontier. Three bidders with 11-value bids and a quit option yield $12^3 = 1728$ branches at the reveal frontier. This is fine for the example library; it is not what one would deploy against a competitive-bidding mechanism with millions of bid levels. Vegas's contract-side bid types are typically small because Gambit needs them small; the deployed contract can carry larger ranges, but then Gambit-side analysis becomes infeasible. This is the explicit tradeoff implied by "the same source for both views".

Move alphabets must be bounded. An action whose parameter has type `int` is rejected by the Gambit backend at compile time. The Solidity backend has no such restriction (it stores an `int256` directly); the Gambit constraint is enforced via a per-backend disable list in the test harness.

Quit appears as a real move. Because the bail-out subgame is a strategic option in Vegas's semantics, the `.efg` contains a `Quit` branch wherever the source action has an or-handler. This is the right thing game-theoretically (it captures the strategic option of timing out), but it does inflate the tree.

Chance nodes. Public randomness (`random`) is emitted as Gambit chance moves with the relevant probabilities read off the source. Each chance event is a node of its own, with the same simultaneity treatment as a player move (chance events on the same frontier appear as a single composite chance move).

8 Other Backends

Past the EVM and Gambit backends, the rest of the family share the EventGraph but address different consumers: a constraint solver (SMT-LIB), a session-typed protocol checker (Scribble), a Bitcoin Lightning channel (for the two-party fragment), a MAID exporter (for the Thrones game-theory workbench), a GraphViz visualisation, and a Coq/Lean deep embedding of the protocol as a dependent typing problem. Each is short enough that we describe them together here.

8.1 SMT-LIB

The SMT backend (`backend/smt/Smt.kt`) encodes the EventGraph as a set of quantifier-free constraints over per-node "done" flags and per-field value variables. A bounded-type field is a finite-sort variable; a guard is its boolean lowering; dependencies become implications $done_B \Rightarrow \bigwedge_{A \in \mathrm{pred}(B)} done_A$. Reachability queries are written as additional constraints asserting that a chosen role gets a payoff at least x.

The result is given to Z3 (or any SMT-LIB-2 solver) for satisfiability. Two principal uses:

- *Witness searches.* "Is there a play that gives role R a payoff $\geq c$?" A satisfying assignment is a concrete history.
- *Coalition queries.* A DQBF encoding is also generated, in which a chosen coalition's variables are existentially quantified and the complement's are universally quantified. The resulting formula is true iff the coalition has a strategy that achieves a property against any response. This is a coarse analogue of best-response analysis useful for sanity checks on small examples.

Z3 is fast on the example library; the encoding is plain enough that it is also useful as a sanity-check on the IR (a guard that nobody can ever satisfy will show up as a falsifying constraint).

8.2 Scribble

Scribble [13] is a session-types implementation that records the legal sequencing of messages in a multi-party protocol. The Scribble backend (`backend/scribble/`) projects the EventGraph onto the public-message subset — skipping commits, retaining reveals and public yields, dropping internal randomness — and emits a global type whose sequencing is the topological order of the public frontier. Recursion in the produced Scribble protocol is empty (the protocol is acyclic by construction).

The output is useful to anyone who wants a static check that their off-chain client library is sending messages in the right order; Scribble's projection toolchain can derive per-role local types from the global protocol.

8.3 Bitcoin / Lightning

A separate backend (`backend/bitcoin/`) targets the two-party Lightning channel for the subset of Vegas games with exactly two strategic roles. Each EventGraph node is implemented as a state in a Bitcoin script state machine; commitments use OP_HASH160 and OP_EQUAL, with state transitions encoded as signed updates of the channel's commitment transaction. The output of the backend is a textual transcript of the Lightning protocol moves needed to realise the game; deploying it on a real Lightning node is out-of-band work.

This backend is the most specialised of the bunch: it does not generalise to more than two roles or to non-balanced deposit structures. Its value is that it demonstrates that the EventGraph abstraction lifts to a non-EVM execution layer with completely different cryptographic primitives, with no change to the IR.

8.4 MAID

MAID [14] (Multi-Agent Influence Diagrams) factor a multi-agent decision problem into decision nodes, chance nodes, utility nodes, and the conditional probability distributions that wire them together. Vegas can emit a MAID suitable for the Thrones game-theory workbench.

A genuine limitation lives here. In plain MAIDs, a decision node's action set is a single static domain; there is no place to express "the legal action set depends on the parents' values". A Vegas guard like `where Host.goat != Guest.d` therefore has nowhere to go in a MAID: the encoding would silently drop the constraint and present the Host as being able to pick any door. The current MAID emitter refuses to emit for any game whose guards reference fields written by other nodes; it accepts self-only guards (a guard that only constrains the value being written) but does not yet narrow the resulting MAID node's domain. Both narrowing self-only guards and supporting context-dependent domains in some MAID extension are listed in the project's `TODO.txt`.

The honest reading is that MAID is a less expressive target than Gambit's EFG for Vegas's class of protocols. Where it *does* apply, the MAID view is more compact than the unfolded tree; for protocols that fit, the output is useful.

8.5 Coq / Lean: Diamond DAG witness records

The Gallina (Coq) and Lean 4 backends emit a deep embedding of the EventGraph as a dependently-typed family of records. The shape of the encoding deserves a paragraph of its own, because it is slightly unusual and corresponds to an original Vegas-specific proof-game idea.

For an EventGraph with n nodes, the backend emits a chain of records $W_0, W_1, \ldots, W_n$, where W_i is the (typed) state after the first i nodes have fired. Each W_i is parameterised on the entire

history $W_0, \ldots, W_{i-1}$, with fields:

- one named field per parameter written by node i, of the parameter's bounded type, with the field name embedding the parameter and owning role (e.g. `hidden_car_Host` for a committed value, `d_Guest` for a public yield);
- one proof field per guard — W_i`_guard_domain_`f for the domain restriction and W_i`_guard_logic` for the user-supplied `where` clause;
- for reveals, a proof W_i`_guard_reveal_`f that the revealed value equals the value committed in an ancestor witness;
- nothing else.

The backend emits one of two top-level records, named for the side of the protocol they encode. `ActionDag` is the *fair-play structure*: an inhabitant is a complete play of the protocol together with proofs of every guard, every domain restriction, and every reveal-matches-commit equation. Inhabiting `ActionDag` is therefore the same as exhibiting that a fair play exists. `EventDag` is the *trace structure*: an inhabitant is a realised history, with hidden fields wrapped in a `Maybe`-style carrier so that traces can be partially observed. Actor strategies are functions from the prior witnesses $W_0, \ldots, W_{i-1}$ (projected to the actor's view) into the appropriate field of W_i, with proofs — so the encoding captures *at the type level* that every move is legal and every reveal matches its commit.

The Coq and Lean backends each take a policy flag selecting which records to emit:

- `FAIR_PLAY` (`lean-fair` / `gallina-fair`) emits `ActionDag` only. Every guard becomes a runtime `decide`/`rfl` obligation in the appropriate W_i.
- `INDEPENDENT` (`lean-independent` / `gallina-independent`) emits `EventDag` only, with hidden fields wrapped in a `Maybe` carrier and an `isPresent` discipline on reveals.
- `MONOTONIC` (`lean-monotone` / `gallina-monotone`) emits both records, with monotonicity guards (W_i`_guard_have_`w_j) certifying that prior commits remain valid for later reveals.

Each policy is exercised against the same example set in the golden-master harness.

The Diamond DAG encoding is independent of the companion VegasCore Lean library described in §10. The two coexist as different approaches to "the same protocol viewed through a proof assistant": Diamond DAG ships a standalone per-program object whose inhabitation is a witness; VegasCore is a reusable library of definitions and theorems. A third Lean backend, emitting a Vegas program as an instance of VegasCore's `WFProgram` type, is planned (and listed in `TODO.txt`) to bridge the two.

The point of this encoding is two-fold. First, it gives a self-contained Lean object that one can pass to a theorem prover without depending on any large semantics library; the encoding *is* the semantics. Second, since it is a function on the prior witnesses, it encodes the player's choice schedule as a strategy in the strict, type-theoretic sense: an inhabitant of the function space. Players' move spaces are subtypes pinned by domain and where-clause proofs. This view is somewhat closer to the IF-logic/CGT [11, 12] style of compositional game semantics than to the state-machine view that Gambit's tree presents.

The Diamond DAG encoding is independent of the companion VegasCore Lean library described in §10. The two coexist as different approaches to "the same protocol viewed through Lean": Diamond DAG ships a per-program standalone object, VegasCore is a reusable library. A planned third backend (also in `TODO.txt`) emits a Vegas program as an instance of VegasCore's `WFProgram` type, completing the picture.

8.6 GraphViz

The simplest backend: a textual DOT description of the EventGraph for debugging and presentation, with node shapes encoding visibility (rounded boxes for PUBLIC, squares for COMMIT, hexagons for REVEAL, diamonds for sinks). Useful for inspecting the result of the commit-reveal expansion pass.

9 Liveness and Quit Handlers

Vegas treats the operational question of *what happens when someone stops responding* as part of the surface syntax rather than as an implementation detail. This section walks through the design: how the language exposes the choice to the programmer, how the compiler realises it on the EVM, and where the model is honest about its current limits.

9.1 Why liveness is in the language

A multi-party on-chain protocol is exposed to two kinds of slow-or-absent participants: strategic ones (who decline to play because the next move is not in their interest), and incapable ones (client crashes, mempool censorship, gas exhaustion). The deployed contract has to decide what to do in both cases, and the decision must be a total function of public history. Vegas makes that decision part of the source: every action that could plausibly fail to occur carries an explicit handler:

```
yield or split A(x: int);
```

The handler describes what the contract does, and the game-theoretic analysis sees `Quit` as a strategic move available to the actor. The two views agree by construction.

9.2 The three group handlers

A statement involving one or more roles in a single `yield/commit/reveal` group carries one of three handlers.

split The quitter's deposit is split among the surviving roles (in a two-player game, the survivor gets it all). This is the default for adversarial games: nobody profits by walking out.

burn The quitter's deposit is sent to a designated burn address (or zero address, depending on chain conventions). Useful in games where giving the deposit to surviving roles would warp incentives — voting schemes where a missing voter should not enrich the others, for instance.

null The missing field is treated as `null` at the typing level (the field's type becomes `opt T`), the node's `done_` flag is set by the contract, and the protocol proceeds. The `withdraw` clause is required to handle the optional case explicitly. Useful for protocols where a non-response is itself strategically meaningful (a delegate failing to delegate).

9.3 Per-query payoff handlers

A single multi-role statement sometimes needs different outcomes depending on which role quit. The language supports this via a `||` qualifier on the role's query:

```
yield or split Bob(ok: bool) || { Alice -> 40 };
```

The right-hand side of `||` is an *outcome*: a payoff allocation chosen from the same sublanguage that `withdraw` clauses use (a brace-block payoff, or `split`/`burn`/`null`). When the named role quits, the contract executes the handler's outcome *instead of* the rest of the protocol; control does not return. No subgame or callable game block is involved — the handler is a terminal allocation chosen at the bail point.

9.4 Operational realisation

The Solidity backend implements `Quit` via a timeout-and-bail mechanism (§6.5). The contract maintains one global inactivity window: if no action has happened anywhere for `TIMEOUT` seconds (default 24 hours), the next caller's modifier marks the named role as `bailed` and resets the window. Once bailed, the role's actions are rejected by the `by(Role)` modifier; the bail is one-way.

This single-window design is operationally simple and implementable in a few lines of Solidity, but its attribution is structural rather than analytical: it marks "the role pointed at by the modifier" as bailed, not "the role that was supposed to act next". In a long protocol with several outstanding actions in different roles, this attribution can be coarser than one would ideally want. A per-action deadline scheme would address this at the cost of substantially more state; the current design is the simpler starting point.

9.5 Liveness in the game model

For the Gambit backend, `Quit` is an additional alphabet symbol at every move site with a handler. Equilibrium analyses therefore see `Quit` as a real move, with the payoff indicated by the handler. This is what lets the analysis surface griefing incentives without modelling them outside the game. In a simultaneous `or split` statement, for instance, `Quit` is strictly dominated by any non-quit move iff the split penalty exceeds the worst non-quit payoff; the dominance check on the generated EFG verifies the property mechanically.

9.6 Limits today

Two known limits worth flagging.

Fair-play liveness is not statically checked. A *fair-play guard* is a `where` clause whose role can always find a satisfying value, regardless of prior moves. The language does not verify that today; a contextual guard like `yield B(y: int) where y != A.x && y != A.x + 1 && y != A.x - 1` over a type `int = {0..2}` becomes unsatisfiable depending on `A`'s play, forcing `B` to quit through no fault of its own. The project's design notes sketch a static check (exhaustive enumeration over bounded types, or an SMT call for the general case) that would type the affected field as `opt T` when satisfiability cannot be established; implementing it is on the TODO list under the name *fair-play liveness*.

Computational tractability is not checked either. A guard such as `p * q == N` for a large semiprime `N` is satisfiable by definition but practically infeasible to satisfy. Vegas treats this the same as a satisfiable guard; the player will quit, the contract will bail, and the game model will record that outcome via the handler. No analysis distinguishes "empty move space" from "intractable move space". This is the standard limit of purely structural reasoning about action availability.

10 Connection to VegasCore

A companion project, VegasCore, formalizes a core calculus over the same event-graph carrier in Lean 4 [9, 26]. The two artifacts are independent — Vegas is a compiler tool; VegasCore is a self-contained semantic construction — but they describe overlapping objects, and a programmer interested in proving properties of a Vegas program will eventually want to cross the bridge between them. This section sketches the correspondence, lists where the two diverge, and gives the gap between today's state and a fully verified pipeline.

10.1 What VegasCore is

VegasCore is a Lean 4 library that formalizes the core `commit/reveal/sample/let/ret` calculus underlying Vegas's source language. Its central object is an `EventGraph` type elaborated from a checked program (`WFProgram`); the well-formedness conditions — visibility, freshness of binders, completeness of reveals, normalization of sample distributions, and at-least-one-legal action — are statically encoded as fields of `WFProgram`. From a checked program VegasCore derives a probabilistic `Machine`, native strategy carriers, and kernel-game presentations of the protocol; it states and proves frontier commutation, the commit/reveal information barrier, agreement of several outcome laws, a finite Kuhn realization theorem, and backend-refinement transport. All of this is mechanized in Lean.

VegasCore's role in the project is to provide a formally-mechanized referent for the kind of semantics that Vegas's compiler informally implements. Where Vegas's IR constructor refuses to accept a graph violating the visibility-on-reads invariant, VegasCore proves the corresponding theorem about the elaborated graph; where Vegas's commit-reveal-expansion pass produces an informationally correct graph by construction, VegasCore proves frontier commutation and the information barrier as theorems about the abstract event graph.

10.2 Shared vocabulary

The two projects use the same names for the things they have in common. The list is short and almost exact:

Vegas (this compiler)	VegasCore (Lean)
EventGraph	EventGraph
node / nodeId	event / node id
guard (`scope`, `expr`)	per-node guard R
COMMIT/ REVEAL/ PUBLICwrites	hidden + commit/reveal node kinds
frontier (`FrontierMachine`)	frontier
`random`	`sample`
`withdraw`	`Ret`

For an entry-level reading of either project, this table is the glossary: the Vegas reader who picks up the VegasCore chapter already knows what an EventGraph is, and vice versa.

10.3 Where they diverge, and why

Each artifact has features the other does not. The asymmetries are deliberate.

Vegas surface keywords and macros. VegasCore's surface language exposes the five core constructors directly; Vegas's surface adds the user-facing `yield/join/commit/reveal/random` keywords, macros for expression-level abbreviation, the `or split/burn/null` handler form, and `||` subgame transfers. These all lower at IR construction time into the core constructors; they exist in Vegas because they make specifications readable, not because they enlarge the semantic universe.

The commit-reveal expansion pass. Vegas's IR runs an explicit rewrite (§5) that introduces commit nodes for concurrent public writes. VegasCore models programs *after* the analogous transformation: its event graph already distinguishes commit and reveal nodes. Vegas's pass is a compiler operation, not a semantic notion. Equivalence of the pre- and post-rewrite graphs at the strategic level is something one would want to state and prove, but as of today that proof is not in either codebase.

Operational subgame layer. Quit handlers, bail-out timeouts, and the resulting subgame structure are first-class in Vegas; VegasCore presently treats quit as a strategic move only, without modelling the deadline mechanism. A backend-refinement chapter in VegasCore covers a generic notion of operational refinement that could carry the bail mechanism, but the Vegas-to-VegasCore mapping of the timeout subgame is not yet written down.

Static obligations. VegasCore's `WFProgram` contains four well-formedness obligations as fields: freshness of binders, reveal completeness, normalized distributions, and at-least-one-legal-action. Vegas's typechecker and IR constructor enforce the same conditions but without exposing them under a single uniform name; they live in the typechecker, in the EventGraph constructor, and in the commit-reveal pass. Lifting them under VegasCore-style names would make the correspondence cleaner; it is on the TODO list.

Backends. Vegas emits Solidity, Vyper, Gambit, SMT-LIB, Scribble, Bitcoin, MAID, GraphViz, and the Diamond DAG witness records (§8.5); VegasCore emits nothing — it is a library of definitions and theorems. This is the kind of asymmetry one expects: one side runs, the other side proves.

10.4 What is and is not proved today

The current state of the two-sided story is the following:

- For programs that lower to VegasCore's core calculus, the semantics formalized by VegasCore is consistent with the EventGraph that Vegas's frontend produces *by construction of the lowering*: the lowering uses the same dependency inference and visibility discipline. The IR-level invariants Vegas's constructor enforces are theorems in VegasCore.
- The various backends agree with Vegas's IR by construction: every backend reads the same EventGraph and lowers it systematically. This is closer to a code-review property than to a formal one — there is no compiler proof.
- There is no formal proof, today, that any specific backend's output (Solidity, Gambit, etc.) refines the EventGraph in any externally meaningful sense. The fellowship-funded extension of this work proposes Solidity-to-EventGraph refinement as the central new theorem on which the practical claim of "what you analyze is what gets deployed" would rest.

- The Diamond DAG (Coq/Lean) backend (§8.5) is a different bet: rather than prove correctness of the compiler once, it ships a typed certificate *with each compiled program*. Both routes — verified compiler and verified compiled artifact — are on the project's TODO list as separate workstreams. They are not redundant; they cover different audiences.

The honest summary: Vegas can be used today as a productive tool without VegasCore (the compiler stands alone), and VegasCore can be used today as a Lean library without Vegas (the calculus stands alone). Connecting them with mechanically-checked refinement proofs is the natural next step and the main subject of ongoing work.

11 Examples and Testing

This section describes the example library, the golden-master test harness, and what we have learned from running them. We make no quantitative claim: the goal is to show that the pipeline covers a range of protocol shapes, not to benchmark against any baseline.

11.1 The example library

The repository contains thirty-nine Vegas specifications under `examples/`. They are written by hand and intended to exercise the language across three axes: textbook game-theory examples, smart-contract patterns, and adversarial constructions.

Classical games. `Prisoners.vg` (Prisoners' Dilemma), `OddsEvens.vg` (matching pennies), `MontyHall.vg` (Monty Hall, with and without chance), `Centipede.vg` (the centipede game), `UltimatumGame.vg`, `RPSLS.vg` (rock-paper-scissors-lizard-Spock), `Dominance.vg` (dominance-solvable example), `TicTacToe.vg`.

Mechanism design. `VickreyAuction.vg` (sealed-bid second-price), `EntryFeeAuction.vg`, `GasPriceAuction.vg`, `HiddenReserve.vg` (reserve-price auction with hidden reserve), `Lottery.vg`, `ThreeWayLottery.vg`, `ThreeWayLotteryBuggy.vg` (an intentionally-flawed variant used to demonstrate the equilibrium analyses catching a vulnerability), `CommitteeVoting.vg`, `StakeAndVote.vg`.

Smart-contract patterns. `EscrowContract.vg` (buyer/seller/arbiter escrow with dispute), `MicropaymentChannel.vg` (two-party payment channel), `AtomicSwap.vg`, `BribedArbitrator.vg` (collusion-resistance test), `ClaimFraud.vg`, `DataAvailability.vg`, `Insurance.vg`, `StakingSlashing.vg`, `PrivacyNoise.vg`.

Adversarial / structural. `Bet.vg` (simple betting), `DoubleFlipLights.vg`, `CongestionRouting.vg`, `RandomLeader.vg`, `RumorForwarding.vg`, `Puzzle.vg`, `SpinTheDial.vg`, `TwoRobotCorridor.vg`.

Most examples are between ten and forty lines of Vegas source. The largest, `TicTacToe.vg`, exercises bounded enumeration of board states. The Vickrey auction is the canonical mechanism example.

11.2 The golden-master harness

The test suite is structured around *golden masters*: expected backend outputs checked into the repository under `src/test/resources/golden-masters/`, with one subdirectory per backend. For

each backend B and each example E for which B is applicable, the test harness compiles E with B and asserts byte-for-byte equality with the golden master `golden-masters/`B`/`E`.ext`. When a regression runs, the actual output is written to `test-diffs/`B`/`E`.ext` and a diff is generated; a small Python helper `copy-test-outputs.py` promotes accepted changes back to the golden masters.

This is the same kind of testing one would do for a compiler that emits any large text artifact: most changes to the generated code are wrong (regressions); the few that are right are accepted explicitly. The harness is the project's primary defense against unintended drift across backends.

Backends are not always applicable to all examples. Three common reasons:

- Gambit and other game-theoretic backends reject any program whose move alphabet is unbounded (`int`); these examples are restricted to the operational backends in the test list.
- Bitcoin/Lightning supports only the two-strategic-role subset; multi-bidder auctions are excluded.
- MAID rejects any guard referencing a field written by another node (§8); guarded examples carry an explicit backend-disable annotation.

Each example has a small declaration listing the backends to disable; the harness uses this list when iterating. Today roughly half of the examples are accepted by every applicable backend in the test list; the other half exercise one or two backend-specific concerns and disable a single backend for a documented reason.

11.3 What we've learned

A few patterns that the example library has surfaced.

Distribution semantics changes the games. Many of the classical game-theory examples cannot be stated literally in Vegas: with a fixed pool, both-defect cannot be made worse than both-cooperate in absolute terms (the pool conserves). The `Prisoners.vg` variant therefore shifts the outcome matrix while preserving the Nash structure. This is not a weakness of the language — it is a structural consequence of the conservation property the language enforces — but it matters for translating textbook problems faithfully.

Guards that look local are often contextual. The Monty Hall `Host.goat != Guest.d` clause is the canonical example: it reads as a local constraint on Host's move, but it references another role's prior write. Programmers writing Vegas tend to underestimate how often this happens. The MAID backend's restriction is what brought this pattern into focus (the MAID encoding ignores it silently).

Liveness handlers are non-trivial design. Choosing between `or split`, `or burn`, and `or null` for any given action is a genuine design decision that affects both the contract and the game. The example library carries explanatory comments around these choices, especially in `OddsEvens.vg`, where the comment block discusses the distinction between losing (a strategic outcome) and quitting (a non-strategic disruption) and the payoffs that make them distinguishable.

Front-running protection is the main payoff for auctions. The Vickrey example, written naively as three simultaneous public `yield`s, is automatically rewritten to a commit-reveal protocol. Reading the Solidity output makes the rewriting visible: each bidder gets a pair of functions `move_`B_i`_c(bytes32 _hidden_b)` for the commit and `move_`B_i`_r(int256 _b, uint256 _salt)` for the reveal (with topological indices c, r), and the reveal's `depends` chain lists every other bidder's commit as a prerequisite. A hand-written equivalent would need careful attention to make this dependency right; here the IR computes it once.

12 Related Work

Vegas sits at the intersection of three lines of work — domain specific languages for smart contracts, automated game-theoretic analysis, and event-structure semantics for concurrent processes. None of these alone covers Vegas's design, and the value of the language design is in their combination.

12.1 Smart-contract domain-specific languages

Several existing DSLs target contract correctness. Marlowe [15] offers a small financial contract calculus for Cardano with formal semantics in Isabelle/HOL; its scope is multi-party financial agreements and its proofs are about safety and conservation, not about strategic properties of the players. Scilla [25] provides a typed intermediate language for Zilliqa with Coq-verified semantics and a strict separation of communication and computation; like Marlowe, its guarantees concern functional behavior of the contract rather than the game it implements. Move [2] introduces linear resource types to make "how much can leave the contract" a property of the type system.

Vegas differs in two structural ways. First, the source language describes a *game played by the contract*, not just a contract, and the compiler emits a game-theoretic artifact (a Gambit `.efg`) alongside the executable code. Second, the language is consciously narrower than these alternatives: no recursion, no token transfers, no arbitrary external calls. This narrowness is what gives the multi-backend story its consistency.

Verification tools that check smart contracts against temporal or safety specifications — VerX [23], KEVM [21], Certora [6] — are complementary. Their question is "does this bytecode satisfy this specification"; Vegas's question is "which specification?" Strategically-aware specifications are exactly what Vegas's Gambit/MAID/SMT artifacts are.

12.2 Game-theoretic analysis of smart contracts

The MEV literature [8, 24] documents extensive strategic activity on existing chains: front-running, sandwich attacks, transaction reordering for profit. These analyses are case studies of deployed contracts, performed after the fact and largely by hand. Mitigation work has typically taken two routes: implementation-level cryptographic defenses for commit-reveal patterns [4] and runtime alternatives such as private transaction pools [27]. Vegas's contribution is to build a specific subclass — observation-before action on concurrent public moves — into the language: the mitigation is a structural property of the generated contract, not a discipline applied by humans.

12.3 Multi-agent influence diagrams and game representations

MAID [14] provides a factored representation of multi-agent decisions as a Bayesian network with decision and utility nodes. We discussed the MAID backend in §8; the relevant limit is that plain MAIDs cannot encode context-dependent action sets, which makes them an awkward target for

Vegas games whose guards are non-trivial. Constrained-MAID extensions exist in the literature but are not standardized.

The Gambit project [19] and OpenSpiel [16] provide tools for analyzing extensive-form games once one is given. Vegas adds the path from a programmer's specification to such a game.

12.4 Event structures and dependency graphs

The EventGraph is a particular flavour of event structure [20, 28] specialized to the protocol setting — typed, finite, visibility-aware, with explicit commit/reveal node kinds. The program-dependence graph [10] and static-single-assignment form [7] are the immediate technical antecedents on the program-analysis side: EventGraph edges are computed by the same dependency analysis those works pioneered, only specialized to the `commit/reveal` discipline.

12.5 Compositional game theory and IF logic

Compositional Game Theory [11] and the work it builds on develop a categorical view of open games that compose like programs. IF logic [12, 18] provides a logical apparatus for imperfect-information games in which informational independence is a first-class construct. Vegas's per-program Diamond DAG encoding (§8.5) has a sympathetic flavour — a player's choice schedule is a function on prior witnesses, which is the IF-logic notion of a Skolem function for an existential under independence — but the connection is at the level of stylistic resonance, not a formal correspondence we have spelled out. The companion VegasCore (§10) does this work on the side of native kernel-game semantics rather than open-game composition.

12.6 Multiparty session types

Multiparty session types [13] specify the legal sequencing of messages in a multi-party protocol; Castellan and Yoshida's recent work [5] relates session types to game semantics directly. Vegas's Scribble backend uses session types as a target, projecting the EventGraph's public-message subset onto a global type. The relationship is more "information set $\subseteq$ session type than the reverse": session types record which messages are legal in a given prefix, but they do not directly record information sets. A fuller comparison is future work.

12.7 Refinement and operational correctness

The bail-out subgame and the Solidity scheduling layer share the shape of an implementation refining a more abstract specification, in the sense of Lynch and Vaandrager [17]. Vegas's companion formalization VegasCore states a stochastic-step refinement theorem on the abstract event-graph machine. A formal proof that Vegas's Solidity backend refines its EventGraph is on the project's TODO list; this is precisely the gap the fellowship proposal addresses.

13 Discussion

13.1 What Vegas is good at today

Vegas is, in its current form, a productive tool for the narrow class of protocols it covers. A specification of a sealed-bid auction, an escrow, a voting scheme, or a small game becomes a file an order of magnitude smaller than the equivalent hand-written Solidity contract, with the strategic

structure explicit and the front-running mitigation automatic. The Gambit, SMT, and witness-record backends provide views that would otherwise be done by hand and seldom done at all. The test harness keeps the views consistent.

The single thing the tool is designed to be good at is making the back and forth between "the protocol I want to play" and "the contract that will play it" short, mechanical, and auditable. The example library exercises this with forty specifications spanning textbook game theory, mechanism design, and adversarial-pattern smart-contract scenarios. The pipeline runs in fractions of a second per example.

13.2 What Vegas is not yet good at

Several limitations are honest enough to acknowledge in detail.

Expressive power. The language is, by design, limited to finite types and acyclic protocols. Loops, dynamic player counts, ERC-20/ERC-777 token flows, and external contract calls are all out of scope. Most real-world protocols use at least some of these. We expect to grow the language; the IR is intended to be robust to those extensions, but the current language is the bounded fragment that lets every backend stay exhaustive.

No mechanical equivalence between backends. The backends each read the EventGraph and emit their own artifact. Equivalence is by construction and by code review, not by proof. A regression that ships an inconsistent pair of artifacts would not be caught by anything stronger than the golden-master test suite, and that suite checks output text rather than semantic agreement. The companion VegasCore (§10) is the route to a mechanical guarantee here, and the Solidity-to-EventGraph refinement proof is the central fellowship-funded continuation of this work.

Fair-play liveness is not statically guaranteed. A contextual guard can become unsatisfiable for fair-playing participants depending on prior moves. The language treats this the same as a strategic quit, which is sometimes the right thing (if the guard's domain genuinely shrunk) and sometimes too permissive (if a fair-playing actor was forced out). A static satisfiability check over bounded types would address the common case; it is on the TODO list under the working name *fair-play liveness*.

Gas costs are not in the model. Vegas's Gambit output ignores gas; equilibrium analyses do not see action costs. For some protocols (sealed-bid auctions with small bid domains) this is the right idealization; for others (high-frequency games where gas is a strategic variable) it is a real omission. Bounded per-action upper bounds derivable by tools like GASTAP [1] could be folded in as action-cost annotations; this is a planned extension.

MAID is gated, not fixed. As detailed in §8, the MAID encoding silently dropped context-dependent guards. The current emitter refuses such inputs. A self-only-guard narrowing pass, or a switch to a constrained-MAID extension, would broaden the MAID backend's applicability.

The Diamond DAG idea is interesting but underexplored. The Coq/Lean witness-record encoding (§8.5) captures a player's strategy as a function on prior witnesses. We have not pushed this far enough to know what it can prove that ordinary tree-shaped semantics cannot. It is, today, an evocative idea: every play is an inhabitant of a dependent record, and every strategy is a typed function on histories.

13.3 Foundations first

The unifying theme of the limitations is intent rather than oversight. Vegas was built with the explicit goal of staying small enough to be honest about: a finite-graph IR, finite-type moves, a fixed pool, deterministic guards, no external calls. This is a smaller language than its closest neighbours in the DSL space (Marlowe, Scilla, Move), and far smaller than general-purpose Solidity. The smallness is what makes the multi-backend story coherent. An EventGraph is exhaustively explorable; an unbounded loop is not. A finite move alphabet yields a finite tree; an unbounded `int` does not.

That smallness is a starting point, not a ceiling. We expect to relax pieces of it as we understand which extensions preserve the analytical properties — bounded iteration (where the bound is statically derivable), token flows (with proven conservation laws), per-round subgames composed from a fixed library. The right way to do these extensions is to add them to the IR carefully and re-derive each backend in turn; the discipline of the current architecture is what makes this plausible.

13.4 Future work

The most immediate work items are recorded in the project's `TODO.txt`. Here we mention the larger themes.

- *Verified compilation and verified compiled artifacts.* Two complementary routes to mechanical guarantees: a once-and for-all proof that the compiler refines the EventGraph semantics for any input (the VegasCore-side proof), and a per-program certificate emitted alongside each artifact (the Diamond DAG route plus a VegasCore-importing backend that emits `WFProgram` values with their well-formedness obligations discharged by elaboration). These cover different audiences: tool authors care about the first, end users about the second.
- *Bounded extensions.* Variable deposits, ERC-20 token flows, and gas-aware liveness (the gas-escrow stage policy sketched in the fellowship proposal) are the natural next steps. Each requires care to keep the analytical backends honest.
- *Operational refinement to the EVM.* The bail-out subgame is the place where the Solidity contract and the game model can most plausibly disagree. A Lean proof that the contract refines its EventGraph under the explicit execution model (ordering adversary, public mempool, fixed timeouts) is the central theorem of the fellowship-funded workstream.
- *Workbench integration.* The Thrones workbench (an external companion project) explores equilibrium analyses interactively on the emitted MAIDs and EFGs. A tighter integration — e.g. replaying the analysis as a back-annotation on the Vegas source, surfacing dominance and best-response findings inline — is plausible.

The goal is not a finished tool today. The goal is a foundation that grows in a disciplined way, where every extension has to keep the multi-backend story honest. We hope the present description is a useful starting point for that conversation.

References

[1] Elvira Albert, Jesús Correas, Pablo Gordillo, Guillermo Román-Díez, and Albert Rubio. GASTAP: A gas analyzer for smart contracts. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 118–125, 2020. doi: 10.1007/978-3-030-45237-7_7.

[2] Sam Blackshear, Evan Cheng, David L. Dill, et al. Move: A language with programmable resources. *Libra Assoc.*, 1, 2019.

[3] Manuel Blum. Coin flipping by telephone: A protocol for solving impossible problems. *ACM SIGACT News*, 15(1):23–27, 1983. doi: 10.1145/1008908.1008911.

[4] Lorenz Breidenbach, Phil Daian, Florian Tramèr, and Ari Juels. Enter the hydra: Towards principled bug bounties and exploit-resistant smart contracts. In *27th USENIX Security Symposium*, pages 1335–1352, 2018.

[5] Simon Castellan and Nobuko Yoshida. Two sides of the same coin: Session types and game semantics. *Proc. ACM Program. Lang.*, 3(POPL), 2019. doi: 10.1145/3290340.

[6] Certora, Inc. The certora prover: Industrial-strength formal verification for smart contracts. Technical report, Certora, 2022.

[7] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 13(4):451–490, 1991. doi: 10.1145/115372.115320.

[8] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 910–927, 2020. doi: 10.1109/SP40000.2020.00043.

[9] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[10] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 9(3):319–349, 1987. doi: 10.1145/24039.24041.

[11] Neil Ghani, Jules Hedges, Viktor Winschel, and Philipp Zahn. Compositional game theory. In *LICS '18*, pages 472–481, 2018. doi: 10.1145/3209108.3209165.

[12] Jaakko Hintikka and Gabriel Sandu. Informational independence as a semantical phenomenon. In *Logic, Methodology and Philosophy of Science VIII*, pages 571–589. North-Holland, 1989.

[13] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *POPL '08*, 2008. doi: 10.1145/1328438.1328472.

[14] Daphne Koller and Brian Milch. Multi-agent influence diagrams for representing and solving games. *Games and Economic Behavior*, 45(1):181–221, 2003.

[15] Pablo Lamela Seijas, Alexander Nemish, David Smith, and Simon Thompson. Marlowe: Implementing and analysing financial contracts on blockchain. In *Financial Cryptography and Data Security (FC Workshops)*, volume 12063 of *LNCS*, pages 496–511. Springer, 2020. doi: 10.1007/978-3-030-54455-3_35.

[16] Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien Perolat, Sriram Srinivasan, et al. OpenSpiel: A framework for reinforcement learning in games. arXiv:1908.09453, 2019.

[17] Nancy Lynch and Frits Vaandrager. Forward and backward simulations: I. untimed systems. *Information and Computation*, 121(2):214–233, 1995. doi: 10.1006/inco.1995.1134.

[18] Allen L. Mann, Gabriel Sandu, and Merlijn Sevenster. *Independence-Friendly Logic: A Game-Theoretic Approach*. Cambridge University Press, 2011.

[19] Richard D. McKelvey, Andrew M. McLennan, and Theodore L. Turocy. Gambit: Software tools for game theory. Technical report, Gambit project, 2006. URL `https://www.gambit-project.org/`.

[20] Mogens Nielsen, Gordon Plotkin, and Glynn Winskel. Petri nets, event structures and domains, part I. *Theoretical Computer Science*, 13(1):85–108, 1981. doi: 10.1016/0304-3975(81)90112-2.

[21] Daejun Park, Yi Zhang, Manasvi Saxena, Philip Daian, and Grigore Roşu. A formal verification tool for ethereum vm bytecode. In *Proc. ESEC/FSE '18*, pages 912–915, 2018.

[22] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology - CRYPTO '91*, volume 576 of *LNCS*, pages 129–140, 1992. doi: 10.1007/3-540-46766-1_9.

[23] Anton Permenev, Dimitar Dimitrov, Petar Tsankov, Dana Drachsler-Cohen, and Martin Vechev. VerX: Safety verification of smart contracts. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1661–1677, 2020. doi: 10.1109/SP40000.2020.00024.

[24] Kaihua Qin, Liyi Zhou, and Arthur Gervais. Quantifying blockchain extractable value: How dark is the forest? In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 198–214, 2022.

[25] Ilya Sergey, Vaivaswatha Nagaraj, Jacob Johannsen, Amrit Kumar, Anton Trunov, and Ken Chan Guan Hao. Safer smart contract programming with Scilla. *Proc. ACM Program. Lang.*, 3(OOPSLA), 2019. doi: 10.1145/3360611.

[26] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

[27] Ben Weintraub, Christof Ferreira Torres, Cristina Nita-Rotaru, and Radu State. A flash(bot) in the pan: Measuring maximal extractable value in private transaction ordering markets. In *Proceedings of the 22nd ACM Internet Measurement Conference (IMC)*, pages 458–473, 2022. doi: 10.1145/3517745.3561448.

[28] Glynn Winskel. Event structures. 255:325–392, 1987. doi: 10.1007/3-540-17906-2_31.